

Automatisation des tests E2E

Playwright, Jira Xray et GitLab CI/CD

De l'analyse des besoins au reporting automatisé

Application testée : TechMart

Réalisé par : Ahmed ELHAMRI

Playwright | Jira Xray | GitLab CI/CD

Table des matières

Introduction générale **3**

1 Présentation de l'application TechMart **4**
1.1 Introduction 4
1.2 Présentation générale de TechMart 4
1.3 Origine de l'application 4
1.4 Fonctionnalités principales 5
1.5 Parcours utilisateur principal 6
1.6 Technologies utilisées dans l'application 6
1.7 Conclusion 6

2 Analyse des besoins et création des User Stories dans Jira **7**
2.1 Introduction 7
2.2 Analyse des besoins 7
2.3 Présentation de Jira 7
2.4 Création des User Stories 8
2.5 Critères d'acceptation 8
2.6 Conclusion 9

3 Rédaction et gestion des cas de test avec Xray **10**
3.1 Introduction 10
3.2 Présentation de Xray 10
3.3 Rédaction des cas de test 10
3.4 Organisation des cas de test 12
3.5 Exécution des tests et suivi des résultats 12
3.6 Conclusion 13

4 Automatisation des tests E2E avec Playwright **14**
4.1 Introduction 14
4.2 Présentation de Playwright 14
4.3 Organisation du projet 14
4.4 Utilisation du Page Object Model 15
4.5 Suites de tests automatisés 15
4.6 Exécution locale des tests 15
4.7 Conclusion 17

5 Intégration continue avec GitLab CI/CD **18**
5.1 Introduction 18
5.2 Présentation de GitLab CI/CD 18

5.3 Configuration du pipeline 19
5.4 Exécution des tests dans le pipeline 19
5.5 Gestion des artefacts 21
5.6 Conclusion 21

6 Reporting automatisé avec JUnit, XML et Xray 22
6.1 Introduction 22
6.2 Génération du rapport JUnit 22
6.3 Préparation du fichier XML pour Xray 23
6.4 Import automatique des résultats dans Xray 23
6.5 Publication du rapport HTML 24
6.6 Conclusion 25

7 Résultats, anomalies détectées et bilan 26
7.1 Introduction 26
7.2 Résultats des tests 26
7.3 Anomalies détectées 27
7.4 Apports du projet 28
7.5 Améliorations possibles 28
7.6 Conclusion 28

Conclusion générale 29

Introduction générale

Ce rapport présente mon projet d'automatisation des tests pour l'application e-commerce TechMart. Il explique les étapes suivies, depuis l'analyse des besoins jusqu'à l'exécution automatique des tests et la génération des rapports.

Ce projet est ma première expérience avec plusieurs outils de test. J'ai utilisé Jira pour créer et organiser les User Stories. J'ai utilisé Xray pour rédiger et suivre les cas de test. J'ai utilisé Playwright pour automatiser les parcours utilisateurs. Enfin, j'ai utilisé GitLab CI/CD pour lancer les tests automatiquement et générer les rapports.

L'objectif du projet est de créer un processus de test simple et organisé. Chaque besoin est lié à des cas de test. Les tests importants sont automatisés avec Playwright. Les résultats sont ensuite envoyés vers Xray grâce aux rapports JUnit au format XML.

Ce rapport est organisé en plusieurs chapitres. Le premier chapitre présente l'application TechMart. Les chapitres suivants expliquent l'analyse des besoins, la création des User Stories dans Jira, la gestion des cas de test avec Xray, l'automatisation avec Playwright, l'intégration continue avec GitLab CI/CD et le reporting automatique des résultats.

Chapitre 1

Présentation de l'application TechMart

1.1 Introduction

Avant de créer les User Stories et les cas de test, il est important de comprendre l'application à tester. Ce chapitre présente TechMart, ses fonctions principales et les parcours disponibles pour les utilisateurs.

TechMart est une application récupérée depuis GitHub et utilisée comme support pour apprendre et pratiquer les tests manuels et les tests automatisés.

1.2 Présentation générale de TechMart

TechMart est une application web de vente en ligne. Elle permet aux utilisateurs de consulter des produits technologiques et de réaliser un achat en ligne.

L'application contient un catalogue de six produits. Chaque produit possède un nom, un prix, une catégorie et une quantité disponible en stock.

TechMart permet aussi aux utilisateurs de créer un compte, de se connecter et de se déconnecter. Après la connexion, la session de l'utilisateur reste active même après l'actualisation de la page.

1.3 Origine de l'application

L'application TechMart n'a pas été développée dans le cadre de ce projet. Elle a été récupérée depuis GitHub et utilisée comme application de démonstration.

Elle a servi de support pour pratiquer les différentes étapes du test logiciel : analyse des besoins, création des User Stories, rédaction des cas de test, automatisation avec Playwright, intégration continue et reporting.

Le travail réalisé dans ce projet concerne donc principalement la stratégie de test, les cas de test créés dans Jira et Xray, l'automatisation des scénarios E2E et l'intégration avec GitLab CI/CD.

Source de l'application : `https://github.com/beaucarnes/software-testing-course/tree/main/sample-app`

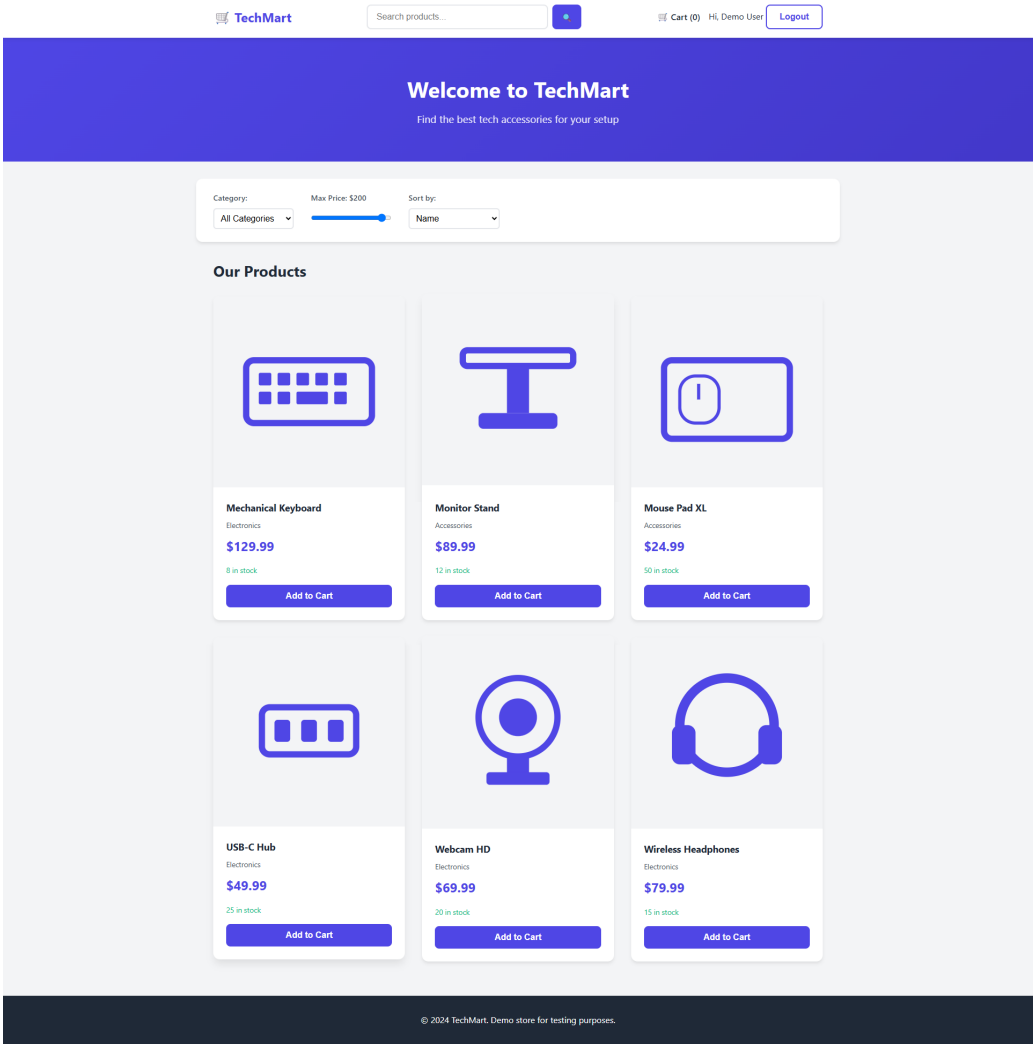


FIGURE 1.1 – Page d'accueil de l'application TechMart.

1.4 Fonctionnalités principales

Les principales fonctionnalités de TechMart sont les suivantes :

— consulter le catalogue des produits ;
— rechercher un produit avec un mot-clé ;
— filtrer les produits par catégorie ou par prix ;
— trier les produits par nom ou par prix ;
— ajouter un produit au panier ;
— modifier la quantité ou supprimer un produit du panier ;
— créer un compte et se connecter ;
— passer une commande avec des informations de livraison et de paiement ;
— vérifier le total, la taxe et la confirmation de commande ;
— mettre à jour le stock après une commande.

Ces fonctionnalités représentent les parcours les plus importants d'une application e-commerce. Elles seront utilisées pour créer les besoins, les User Stories et les cas de test.

1.5 Parcours utilisateur principal

Le parcours principal commence par la consultation du catalogue. L'utilisateur peut rechercher un produit ou utiliser les filtres pour trouver un article.

Ensuite, il peut ajouter un ou plusieurs produits au panier. Dans le panier, il peut modifier la quantité, supprimer un article ou vérifier le montant total.

Enfin, l'utilisateur peut accéder à la page de commande. Il doit compléter les informations de livraison et de paiement. Après la validation, l'application affiche une confirmation de commande et met à jour le stock disponible.

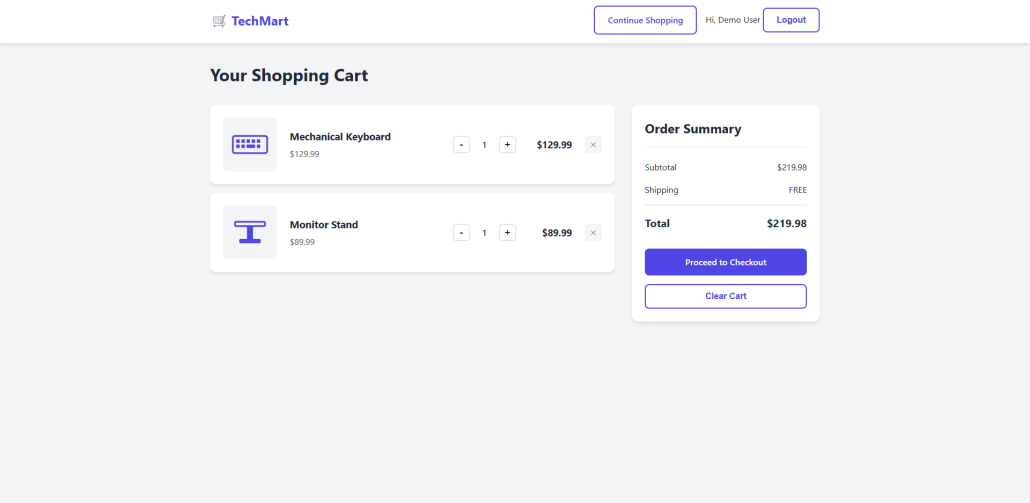


FIGURE 1.2 – Exemple du parcours d'achat dans TechMart.

1.6 Technologies utilisées dans l'application

TechMart utilise une interface web créée avec HTML, CSS et JavaScript. Le serveur est développé avec Node.js et Express.

Le serveur gère les produits, les utilisateurs, les sessions, les paniers et les commandes. Les données sont stockées en mémoire. Cette solution est adaptée à un projet de formation, car elle permet de tester l'application dans un environnement simple et contrôlé.

1.7 Conclusion

Dans ce chapitre, nous avons présenté l'application TechMart et ses principales fonctionnalités. Cette application permet de tester plusieurs parcours importants, comme la recherche de produits, la gestion du panier, la connexion et la commande.

Le chapitre suivant présente l'analyse des besoins et la création des User Stories dans Jira.

Chapitre 2

Analyse des besoins et création des User Stories dans Jira

2.1 Introduction

Après la présentation de TechMart, la première étape du projet consiste à analyser les besoins de l'application. Cette étape permet de comprendre ce que les utilisateurs doivent pouvoir faire et de définir les fonctionnalités à tester.

Les besoins ont ensuite été organisés sous forme de User Stories dans Jira. Ces User Stories servent de base pour créer les cas de test dans Xray.

2.2 Analyse des besoins

L'analyse des besoins a été réalisée à partir des principales fonctions de TechMart. Les parcours les plus importants ont été sélectionnés afin de couvrir les actions d'un utilisateur sur l'application.

Les besoins identifiés concernent principalement :

— la consultation et la recherche de produits ;
— le filtrage et le tri du catalogue ;
— la gestion du panier ;
— la création de compte et la connexion ;
— la validation d'une commande ;
— la mise à jour du stock après un achat.

Cette analyse permet de définir clairement ce qui doit être vérifié pendant les tests. Elle aide aussi à éviter d'oublier des fonctions importantes.

2.3 Présentation de Jira

Jira est un outil de gestion de projet. Il permet de créer, organiser et suivre les besoins d'un projet.

Dans ce projet, Jira a été utilisé pour créer les User Stories. Chaque User Story décrit une fonction attendue par l'utilisateur. Elle contient un titre, une description et des critères d'acceptation.

Le choix de Jira est utile car il facilite l'organisation des besoins et assure une bonne traçabilité entre les besoins, les tests et les résultats.

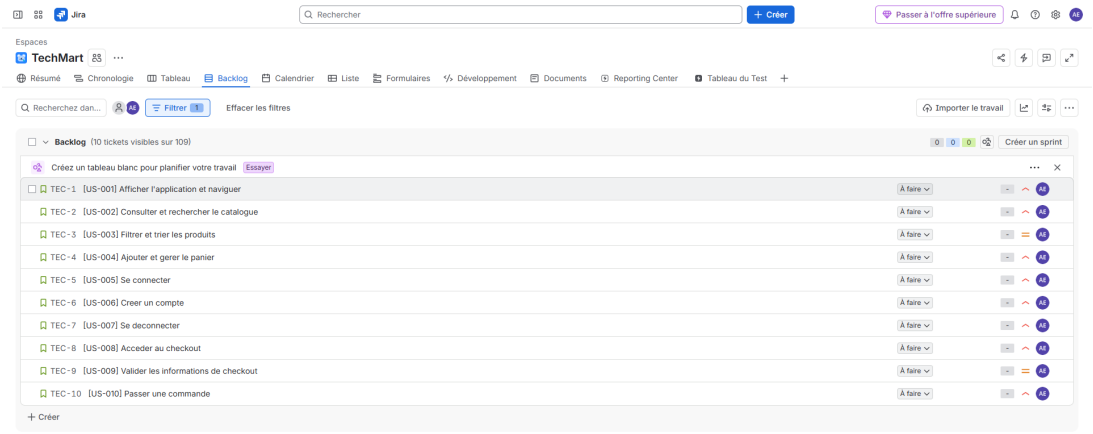


FIGURE 2.1 – Backlog du projet dans Jira.

2.4 Création des User Stories

Les User Stories ont été créées à partir des parcours principaux de l'application TechMart. Elles suivent la forme suivante :

En tant que [utilisateur], je veux [action] afin de [objectif].

Par exemple, une User Story peut être écrite ainsi :

En tant que client, je veux ajouter un produit au panier afin de pouvoir passer une commande.

D'autres User Stories ont été créées pour les fonctions importantes, comme la recherche de produits, la connexion, la création de compte et le paiement.

Fonction	**Exemple de User Story**
Catalogue	En tant que client, je veux rechercher un produit afin de le trouver rapidement.
Panier	En tant que client, je veux modifier la quantité d'un article afin d'adapter ma commande.
Authentification	En tant qu'utilisateur, je veux me connecter afin d'accéder à mon compte.
Commande	En tant que client, je veux valider ma commande afin de terminer mon achat.

TABLE 2.1 – Exemples de User Stories créées pour TechMart.

2.5 Critères d'acceptation

Pour chaque User Story, des critères d'acceptation ont été définis. Ils permettent de préciser le résultat attendu et de savoir si la fonction répond au besoin.

Par exemple, pour la User Story sur le panier, les critères d'acceptation peuvent être les suivants :

— l'utilisateur peut ajouter un produit au panier ;
— le nombre d'articles dans le panier est mis à jour ;
— le sous-total est calculé correctement ;
— l'utilisateur peut modifier la quantité d'un article ;
— l'utilisateur peut supprimer un article du panier.

Ces critères seront utilisés dans le chapitre suivant pour créer les cas de test dans Xray.

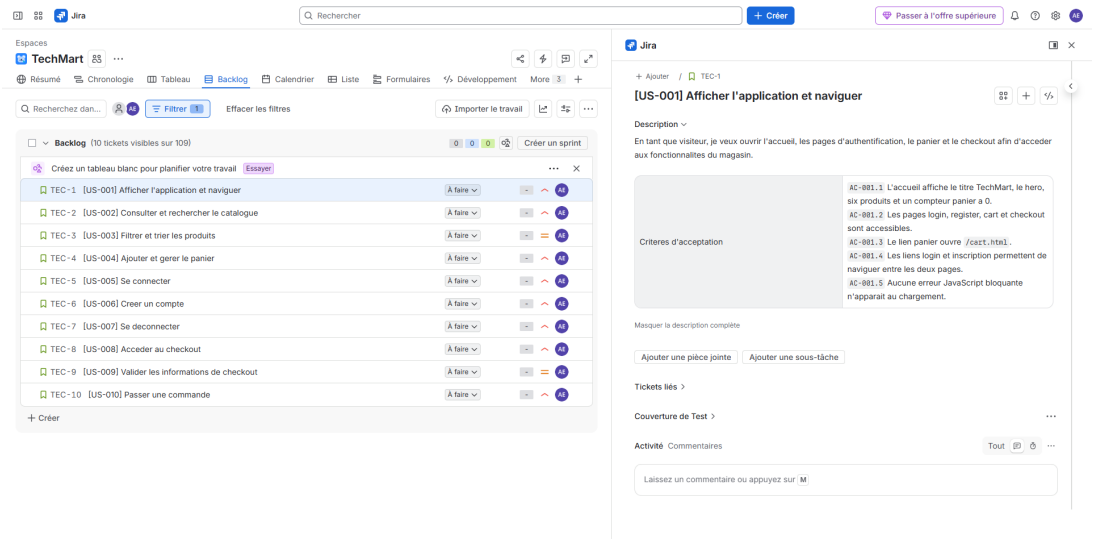


FIGURE 2.2 – Exemple d'une User Story et de ses critères d'acceptation dans Jira.

2.6 Conclusion

Dans ce chapitre, les besoins principaux de TechMart ont été analysés et organisés sous forme de User Stories dans Jira. Cette étape permet de structurer le projet et de définir les fonctions à tester.

Le chapitre suivant présente la création et la gestion des cas de test dans Xray à partir des User Stories définies.

Chapitre 3

Rédaction et gestion des cas de test avec Xray

3.1 Introduction

Après la création des User Stories dans Jira, l'étape suivante est la création des cas de test. Les cas de test permettent de vérifier que chaque fonctionnalité répond bien aux besoins définis.

Dans ce projet, Xray a été utilisé pour créer, organiser et exécuter les cas de test. Xray est directement intégré à Jira. Cette intégration facilite le lien entre les User Stories, les cas de test, les anomalies et les résultats d'exécution.

3.2 Présentation de Xray

Xray est une extension de Jira dédiée à la gestion des tests. Il permet de créer des cas de test manuels et automatisés, de les organiser dans des plans de test et de suivre leurs résultats.

Le choix de Xray permet de centraliser les informations du projet dans Jira. Les besoins, les tests et les anomalies restent dans le même environnement.

Dans ce projet, Xray a été utilisé pour :

— créer les cas de test ;
— définir les étapes à suivre ;
— ajouter les résultats attendus ;
— définir la priorité des tests ;
— organiser les tests dans un Test Plan ;
— exécuter les tests dans une Test Execution ;
— suivre les résultats *PASS*, *FAIL* ou *TODO*.

3.3 Rédaction des cas de test

Chaque cas de test a été créé à partir d'une User Story et de ses critères d'acceptation. Les cas de test décrivent les actions à réaliser et le résultat attendu.

Un cas de test contient généralement les éléments suivants :

— un identifiant de test ;
— un titre clair ;

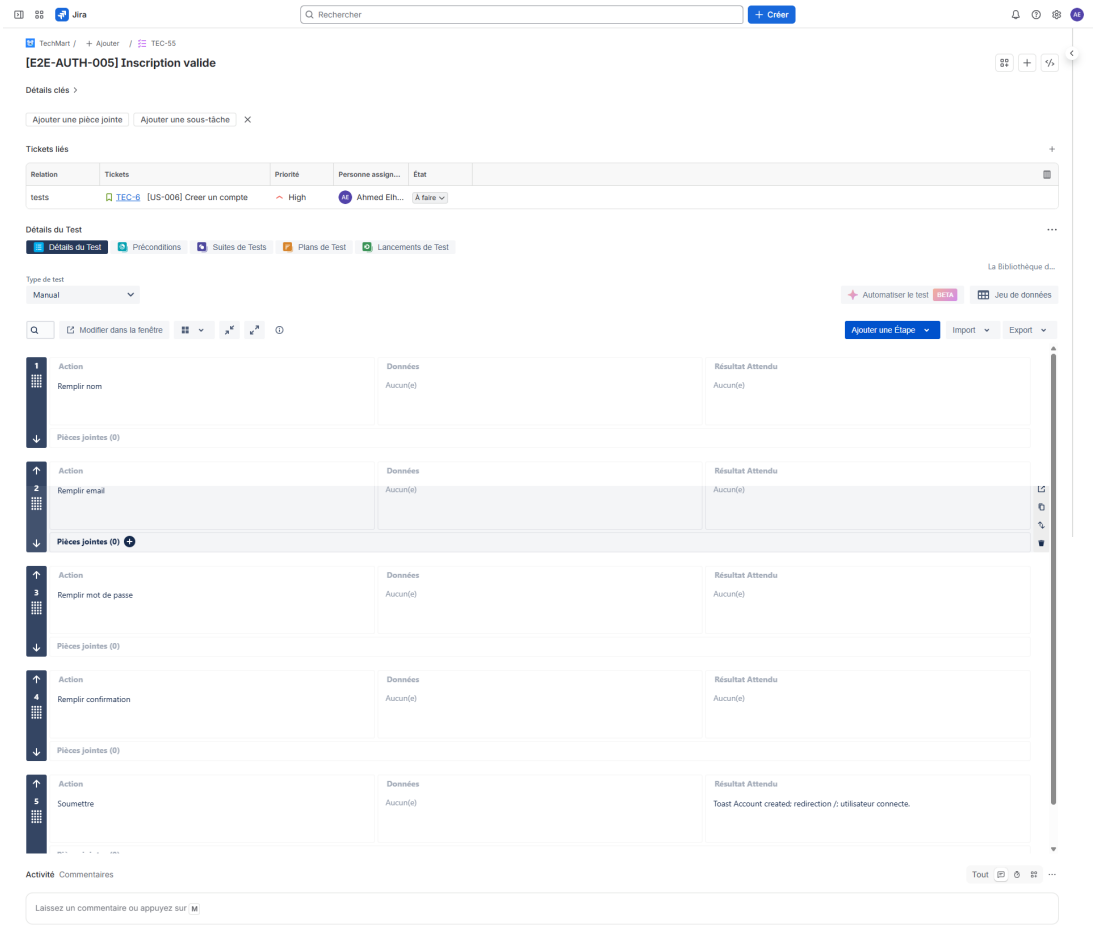


FIGURE 3.1 – Exemple d'un cas de test créé dans Xray.

— une description ;
— des préconditions ;
— des étapes de test ;
— les résultats attendus ;
— une priorité.

Par exemple, pour vérifier l'ajout d'un produit au panier, le cas de test peut être organisé comme suit :

Étape	**Action**	**Résultat attendu**
1	Ouvrir la page d'accueil.	Le catalogue est affiché.
2	Cliquer sur le bouton d'ajout d'un produit.	Le produit est ajouté au panier.
3	Ouvrir le panier.	Le produit et son prix sont affichés.

TABLE 3.1 – Exemple d'étapes pour un cas de test panier.

3.4 Organisation des cas de test

Les cas de test ont été organisés selon les principales fonctions de TechMart. Cette organisation facilite la lecture et l'exécution des tests.

Les groupes de tests créés sont les suivants :

— tests Smoke ;
— tests Catalogue ;
— tests Panier ;
— tests Authentification ;
— tests Checkout.

Les tests Smoke vérifient les fonctions principales de l'application. Les autres groupes couvrent les fonctions plus détaillées du catalogue, du panier, de l'authentification et de la commande.

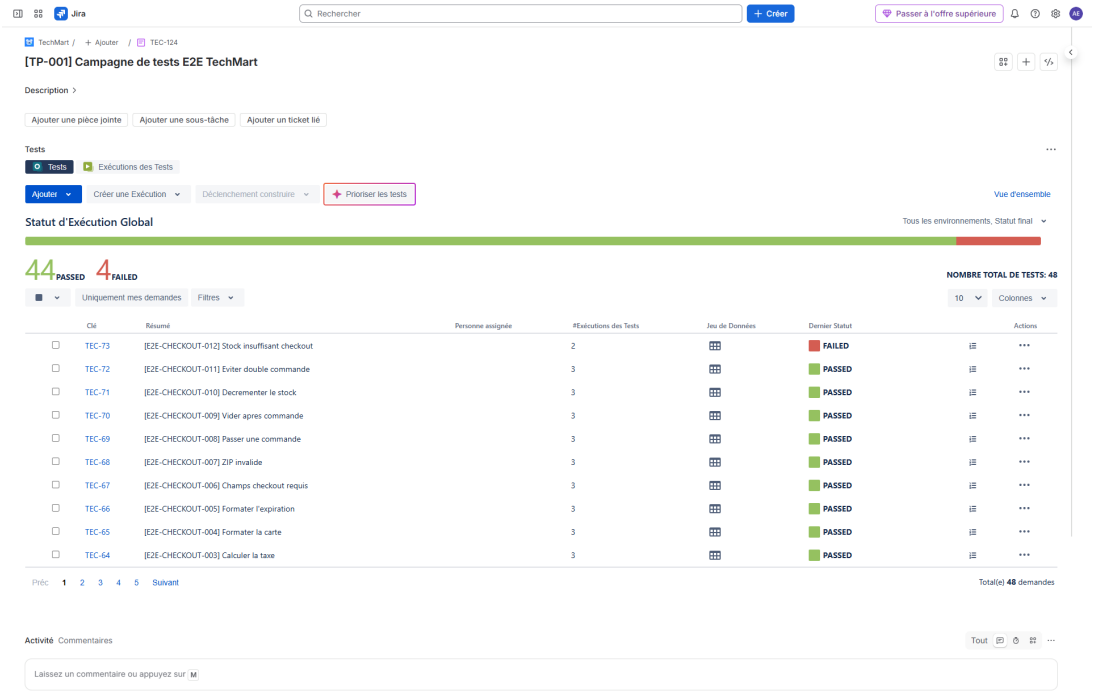


FIGURE 3.2 – Organisation des cas de test dans un Test Plan Xray.

3.5 Exécution des tests et suivi des résultats

Xray permet de créer une Test Execution. Une Test Execution représente une session d'exécution des cas de test.

Pendant l'exécution, chaque test reçoit un statut :

— **PASS** : le résultat obtenu correspond au résultat attendu ;
— **FAIL** : le résultat obtenu est différent du résultat attendu ;
— **TODO** : le test n'a pas encore été exécuté ;
— **EXECUTING** : le test est en cours d'exécution.

En cas d'échec, une anomalie peut être créée dans Jira et liée au cas de test. Cette méthode permet de garder une trace claire des problèmes détectés.

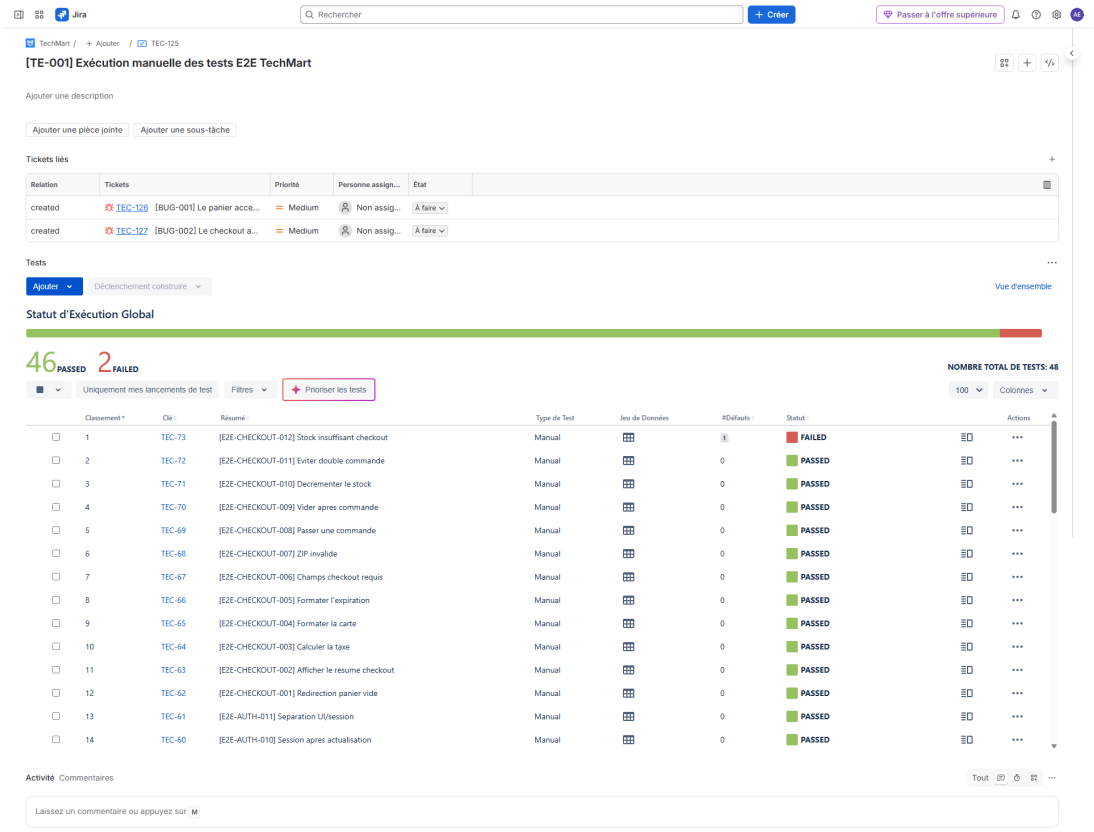


FIGURE 3.3 – Résultats de l'exécution manuelle des tests E2E dans Xray.

3.6 Conclusion

Dans ce chapitre, les cas de test ont été créés et organisés avec Xray à partir des User Stories définies dans Jira. Xray a permis de suivre les résultats des tests et de garder un lien entre les besoins, les cas de test et les anomalies.

Le chapitre suivant présente l'automatisation des cas de test avec Playwright.

Chapitre 4

Automatisation des tests E2E avec Playwright

4.1 Introduction

Après la création des cas de test dans Xray, les scénarios les plus importants ont été automatisés avec Playwright. L'automatisation permet de lancer les tests plus rapidement et de vérifier les fonctions de l'application de manière répétée.

Ce chapitre présente Playwright, son utilisation dans le projet et l'organisation des tests automatisés.

4.2 Présentation de Playwright

Playwright est un outil d'automatisation des tests web. Il permet de simuler les actions d'un utilisateur dans un navigateur, par exemple ouvrir une page, cliquer sur un bouton, remplir un formulaire ou vérifier un message.

Playwright a été choisi pour plusieurs raisons :

— il permet de tester une application web de bout en bout ;
— il est compatible avec plusieurs navigateurs ;
— il permet d'écrire les tests en JavaScript ;
— il génère des rapports de test ;
— il peut être utilisé dans un pipeline GitLab CI/CD.

Dans ce projet, Playwright permet de vérifier les parcours principaux de l'application TechMart.

4.3 Organisation du projet

Les tests automatisés sont organisés dans plusieurs dossiers. Le dossier `tests/` contient les scénarios de test. Le dossier `pages/` contient les Page Objects. Le dossier `data/` contient les données utilisées pendant les tests.

Cette organisation améliore la lisibilité du projet et facilite la maintenance des tests.

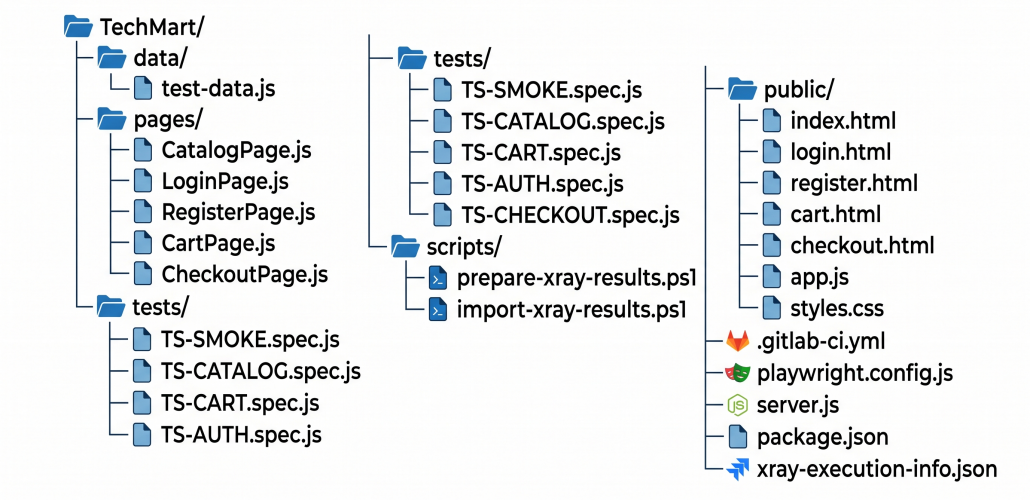


FIGURE 4.1 – Organisation du projet d'automatisation Playwright.

4.4 Utilisation du Page Object Model

Le projet utilise le modèle *Page Object Model*, aussi appelé POM. Ce modèle permet de séparer les actions réalisées sur les pages et les scénarios de test.

Chaque page importante de TechMart possède son propre fichier :

— `CatalogPage.js` pour le catalogue ;
— `LoginPage.js` pour la connexion ;
— `RegisterPage.js` pour l'inscription ;
— `CartPage.js` pour le panier ;
— `CheckoutPage.js` pour la commande.

Par exemple, les actions liées au panier, comme ajouter un article, modifier une quantité ou supprimer un produit, sont regroupées dans le fichier `CartPage.js`.

Cette structure évite de répéter les mêmes actions dans plusieurs fichiers de test.

4.5 Suites de tests automatisés

Les tests sont répartis dans cinq suites principales :

— **Smoke** : vérifie que les pages principales sont accessibles ;
— **Catalogue** : vérifie la recherche, les filtres et le tri des produits ;
— **Panier** : vérifie l'ajout, la modification et la suppression des articles ;
— **Authentification** : vérifie l'inscription, la connexion et la déconnexion ;
— **Checkout** : vérifie la commande, le paiement, la taxe et le stock.

Au total, le projet contient 48 tests E2E automatisés. Ces tests couvrent les fonctions principales de TechMart.

4.6 Exécution locale des tests

Les tests peuvent être exécutés localement avec la commande suivante :

```
npx playwright test
```

```
test.describe('TS-SMOKE', () => {

  test('[TEC-14] E2E-SMOKE-001 - Charger l\'accueil', async ({ page }) => {
    const catalogPage = new CatalogPage(page);

    await catalogPage.open();
    await catalogPage.verifyCatalogPageIsDisplayed();
    await catalogPage.verifyCartCount("0");
    await catalogPage.verifyProductsAreDisplayed();
  });

  test('[TEC-26] E2E-SMOKE-002 - Ouvrir les pages principales', async ({ page }) => {
    const catalogPage = new CatalogPage(page);
    const loginPage = new LoginPage(page);
    const registerPage = new RegisterPage(page);
    const cartPage = new CartPage(page);
    const checkoutPage = new CheckoutPage(page);

    await loginPage.open();
    await loginPage.verifyLoginPageIsDisplayed();

    await registerPage.open();
    await registerPage.verifyRegisterPageIsDisplayed();

    await catalogPage.open();
    await catalogPage.addProductToCart('id', 3);

    await cartPage.open();
    await cartPage.verifyCartPageIsDisplayed();
    await cartPage.proceedToCheckout();

    await checkoutPage.open();
    await checkoutPage.verifycheckoutPageIsDisplayed();

  });

  test('[TEC-27] E2E-SMOKE-003 - Naviguer vers le panier', async ({ page }) => {

    const catalogPage = new CatalogPage(page);
    const cartPage = new CartPage(page);

    await catalogPage.open();
    await catalogPage.goToCart();
    await cartPage.verifyCartPageIsDisplayed();

  });


  test('[TEC-28] E2E-SMOKE-004 - Naviguer login/register', async ({ page }) => {

    const catalogPage = new CatalogPage(page);

    await catalogPage.open();

    await catalogPage.goToLoginPage();
    await catalogPage.verifyLoginPageUrl();

    await catalogPage.goToRegisterPage();
    await catalogPage.verifyRegisterPageUrl();
  });
});
```

FIGURE 4.2 – Exemple d'un test automatisé avec Playwright.

Playwright lance les tests dans le navigateur configuré dans le projet. À la fin de l'exécution, il affiche le nombre de tests réussis et échoués.

Un rapport HTML est également généré. Ce rapport permet de voir le détail de chaque test et d'analyser les éventuels échecs.

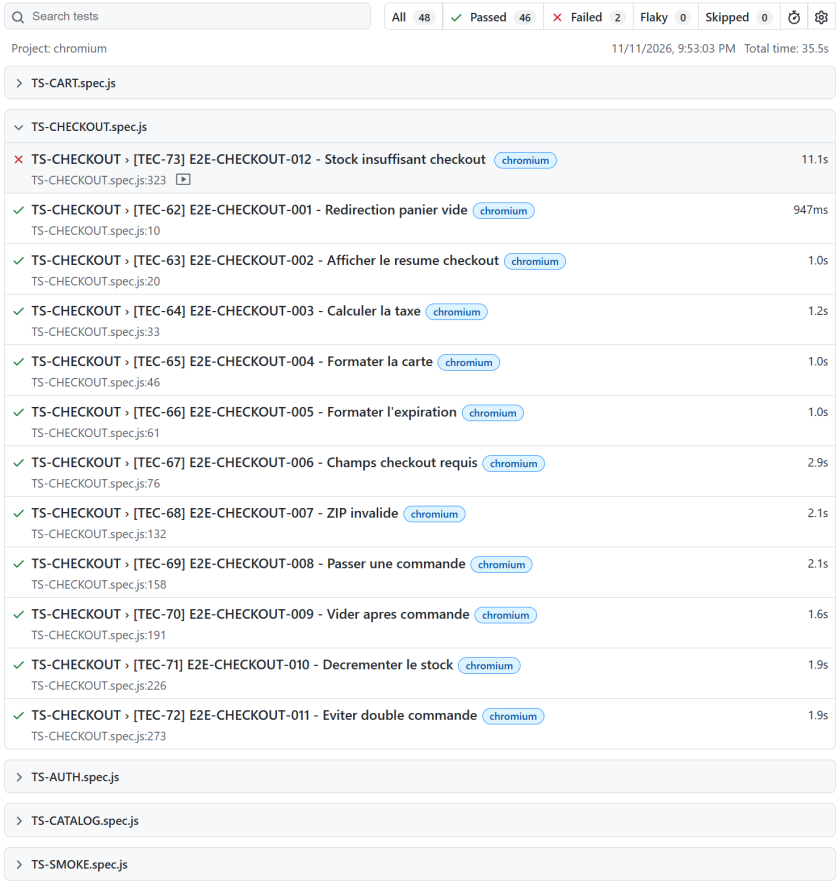


FIGURE 4.3 – Résultat d'une exécution locale des tests Playwright.

4.7 Conclusion

Dans ce chapitre, les cas de test ont été automatisés avec Playwright. L'utilisation du modèle Page Object Model a permis d'organiser les actions liées aux différentes pages de TechMart.

Les tests automatisés couvrent les principaux parcours de l'application. Le chapitre suivant présente l'intégration de ces tests dans un pipeline GitLab CI/CD.

Chapitre 5

Intégration continue avec GitLab CI/CD

5.1 Introduction

Après l'automatisation des tests avec Playwright, il est important de pouvoir les lancer automatiquement. Cette étape est appelée intégration continue.

Dans ce projet, GitLab CI/CD a été utilisé pour exécuter les tests Playwright dans un pipeline. Le pipeline permet de préparer l'environnement, lancer les tests et conserver les rapports générés.

5.2 Présentation de GitLab CI/CD

GitLab CI/CD est un outil intégré à GitLab. Il permet d'automatiser certaines tâches d'un projet, par exemple l'installation des dépendances, l'exécution des tests et la publication des rapports.

GitLab CI/CD a été choisi car il permet de centraliser le code source, les pipelines et les résultats des tests dans le même espace de travail.

Dans ce projet, GitLab CI/CD permet de :

— installer les dépendances du projet ;
— installer le navigateur utilisé par Playwright ;
— lancer les tests automatisés ;
— générer les rapports de test ;
— conserver les fichiers générés comme artefacts ;
— publier le rapport HTML Playwright avec GitLab Pages.

La figure suivante montre plusieurs pipelines du projet. Certains pipelines sont en statut *Failed*. Cet échec est volontaire : des cas de test ont été configurés pour échouer afin de vérifier que le pipeline peut détecter les tests en échec et générer les rapports correspondants.

Même si le stage des tests échoue, les étapes de reporting peuvent continuer. Cette configuration permet de conserver les rapports JUnit, les fichiers XML et les résultats nécessaires pour analyser les échecs et les envoyer vers Xray.

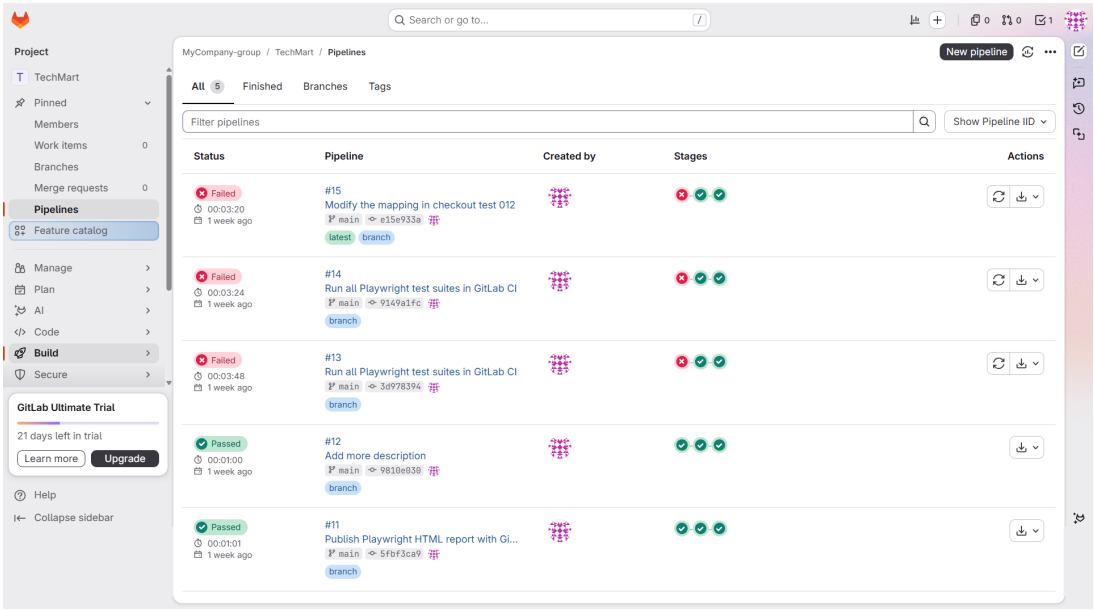


FIGURE 5.1 – Pipelines GitLab : les échecs volontaires permettent de vérifier le reporting des tests échoués.

5.3 Configuration du pipeline

Le pipeline est défini dans le fichier `.gitlab-ci.yml`. Ce fichier contient les étapes qui doivent être exécutées par GitLab.

Le pipeline commence par l'installation des dépendances avec la commande suivante :

```
npm ci
```

Ensuite, le navigateur Chromium est installé pour permettre à Playwright d'exécuter les tests.

```
npx playwright install chromium
```

Enfin, Playwright lance toutes les suites de test du projet.

```
npx playwright test
```

Cette configuration permet d'exécuter les tests dans un environnement propre à chaque lancement du pipeline.

5.4 Exécution des tests dans le pipeline

Lorsqu'un pipeline est lancé, GitLab exécute les différentes commandes définies dans le fichier de configuration.

Le runner GitLab prépare l'environnement, télécharge les dépendances nécessaires et lance les tests Playwright. À la fin de l'exécution, le statut du pipeline indique si les tests ont réussi ou échoué.

Cette exécution automatique permet de vérifier rapidement que les principales fonctionnalités de TechMart continuent de fonctionner après une modification du projet.

```yaml
stages:
  - test
  - xray
  - deploy

playwright-tests:
  stage: test
  tags:
    - windows
    - playwright
  script:
  - npm ci
  - npx playwright install chromium
  - |
      Remove-Item "junit-results.xml" -ErrorAction SilentlyContinue
      ...
      exit $testExitCode
  artifacts:
    when: always
    paths:
      - playwright-report/
      - junit-results.xml
      - xray-junit-results.xml
    reports:
      junit: junit-results.xml
    expire_in: 30 days

xray-import:
  stage: xray
  tags:
    - windows
    - playwright
  when: always
  dependencies:
    - playwright-tests
  script:
    - powershell -ExecutionPolicy Bypass -File scripts/import-xray-results.ps1

pages:
  stage: deploy
  tags:
    - windows
    - playwright
  when: always
  dependencies:
    - playwright-tests
  script:
    - |
      if (-not (Test-Path "playwright-report/index.html")) {
      ...
      Write-Host "Rapport Playwright prêt pour publication."
  pages:
    publish: public
  artifacts:
    paths:
      - public/
```

FIGURE 5.2 – Extrait du fichier de configuration `.gitlab-ci.yml`.

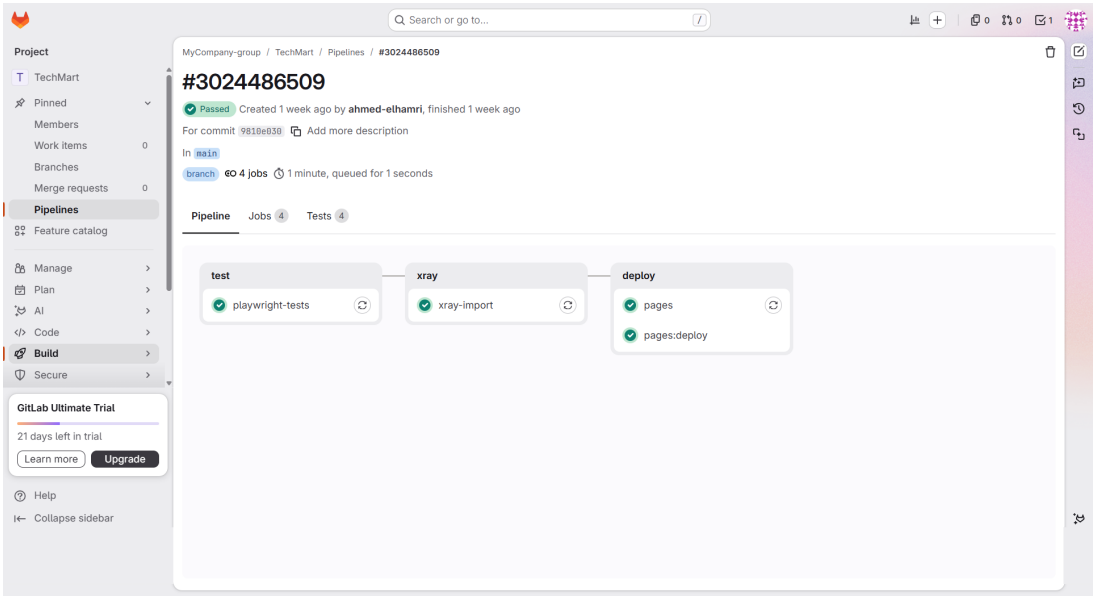


FIGURE 5.3 – Pipeline GitLab réussi : exécution des tests Playwright, import des résultats dans Xray et publication du rapport HTML.

5.5 Gestion des artefacts

Les artefacts sont les fichiers générés pendant l'exécution du pipeline. Dans ce projet, les principaux artefacts sont :

— le rapport JUnit au format XML ;
— le rapport Playwright au format HTML ;
— le fichier XML préparé pour Xray ;
— les résultats et les traces des tests en cas d'échec.

Ces fichiers peuvent être téléchargés depuis GitLab après l'exécution du pipeline. Ils permettent d'analyser les résultats des tests et de préparer leur envoi vers Xray.

5.6 Conclusion

Dans ce chapitre, GitLab CI/CD a été utilisé pour automatiser l'exécution des tests Playwright. Le pipeline permet de lancer les tests, de conserver les rapports et de publier le rapport HTML.

Le chapitre suivant présente la génération des rapports JUnit et XML, ainsi que l'import automatique des résultats dans Xray.

Chapitre 6

Reporting automatisé avec JUnit, XML et Xray

6.1 Introduction

L'exécution des tests ne suffit pas. Il est aussi nécessaire de créer des rapports pour analyser les résultats et suivre la qualité de l'application.

Dans ce projet, les résultats Playwright sont générés au format JUnit. Ils sont ensuite préparés dans un fichier XML compatible avec Xray, puis importés automatiquement dans Jira.

6.2 Génération du rapport JUnit

JUnit est un format de rapport utilisé par de nombreux outils de test. Il enregistre les informations principales d'une exécution, comme le nombre de tests réussis, échoués ou ignorés.

Après l'exécution des tests Playwright, un fichier nommé `junit-results.xml` est généré. Ce fichier contient le résultat de chaque test exécuté.

Le format JUnit a été choisi car il est simple, connu et compatible avec GitLab CI/CD et Xray.

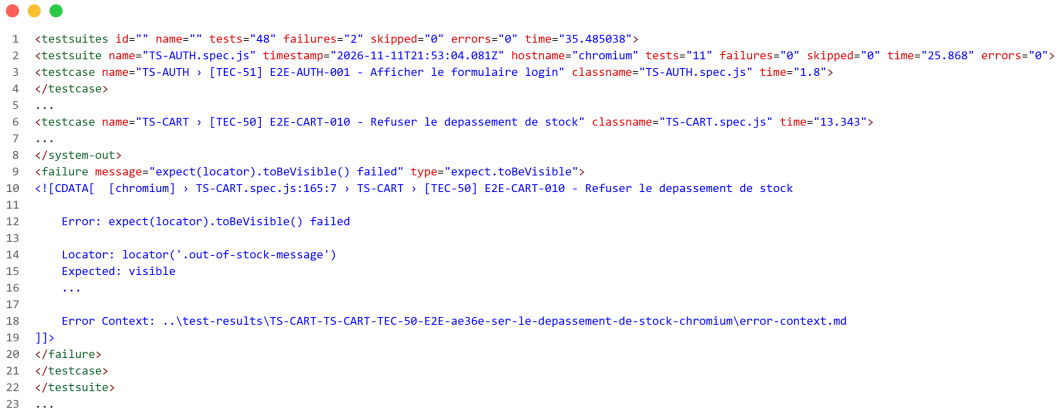


FIGURE 6.1 – Exemple du rapport JUnit généré après l'exécution des tests Playwright.

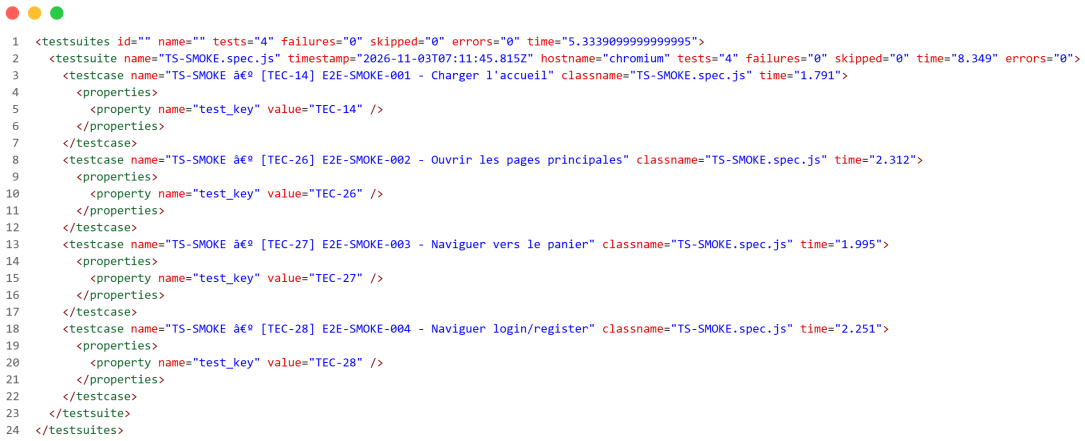
6.3 Préparation du fichier XML pour Xray

Le rapport JUnit généré par Playwright ne contient pas directement toutes les informations nécessaires pour associer chaque résultat à un cas de test Xray.

Un script PowerShell, nommé `prepare-xray-results.ps1`, a été utilisé pour préparer le fichier XML. Ce script récupère les clés Jira présentes dans les noms des tests, par exemple `TEC-14`.

Ensuite, le script ajoute ces clés dans le fichier XML avec la propriété `test_key`. Le fichier généré est nommé `xray-junit-results.xml`.

Cette étape permet à Xray d'identifier le cas de test correspondant à chaque résultat Playwright.



```xml
<testsuites id="" name="" tests="4" failures="0" skipped="0" errors="0" time="5.3339099999999995">
  <testsuite name="TS-SMOKE.spec.js" timestamp="2026-11-03T07:11:45.815Z" hostname="chromium" tests="4" failures="0" skipped="0" time="8.349" errors="0">
    <testcase name="TS-SMOKE â€º [TEC-14] E2E-SMOKE-001 - Charger l'accueil" classname="TS-SMOKE.spec.js" time="1.791">
      <properties>
        <property name="test_key" value="TEC-14" />
      </properties>
    </testcase>
    <testcase name="TS-SMOKE â€º [TEC-26] E2E-SMOKE-002 - Ouvrir les pages principales" classname="TS-SMOKE.spec.js" time="2.312">
      <properties>
        <property name="test_key" value="TEC-26" />
      </properties>
    </testcase>
    <testcase name="TS-SMOKE â€º [TEC-27] E2E-SMOKE-003 - Naviguer vers le panier" classname="TS-SMOKE.spec.js" time="1.995">
      <properties>
        <property name="test_key" value="TEC-27" />
      </properties>
    </testcase>
    <testcase name="TS-SMOKE â€º [TEC-28] E2E-SMOKE-004 - Naviguer login/register" classname="TS-SMOKE.spec.js" time="2.251">
      <properties>
        <property name="test_key" value="TEC-28" />
      </properties>
    </testcase>
  </testsuite>
</testsuites>
```

FIGURE 6.2 – Fichier XML préparé avec les clés des tests Xray.

6.4 Import automatique des résultats dans Xray

Après la préparation du fichier XML, un second script PowerShell, nommé `import-xray-results.ps` envoie les résultats vers Xray Cloud.

Le script utilise l'API Xray pour créer une Test Execution et importer les résultats. Chaque cas de test reçoit alors son statut dans Xray :

— **PASSED** si le test a réussi ;
— **FAILED** si le test a échoué ;
— **TODO** si le test n'a pas été exécuté.

Les résultats automatisés sont ainsi visibles dans Jira, avec les mêmes cas de test qui ont été créés pendant la phase de test manuel.

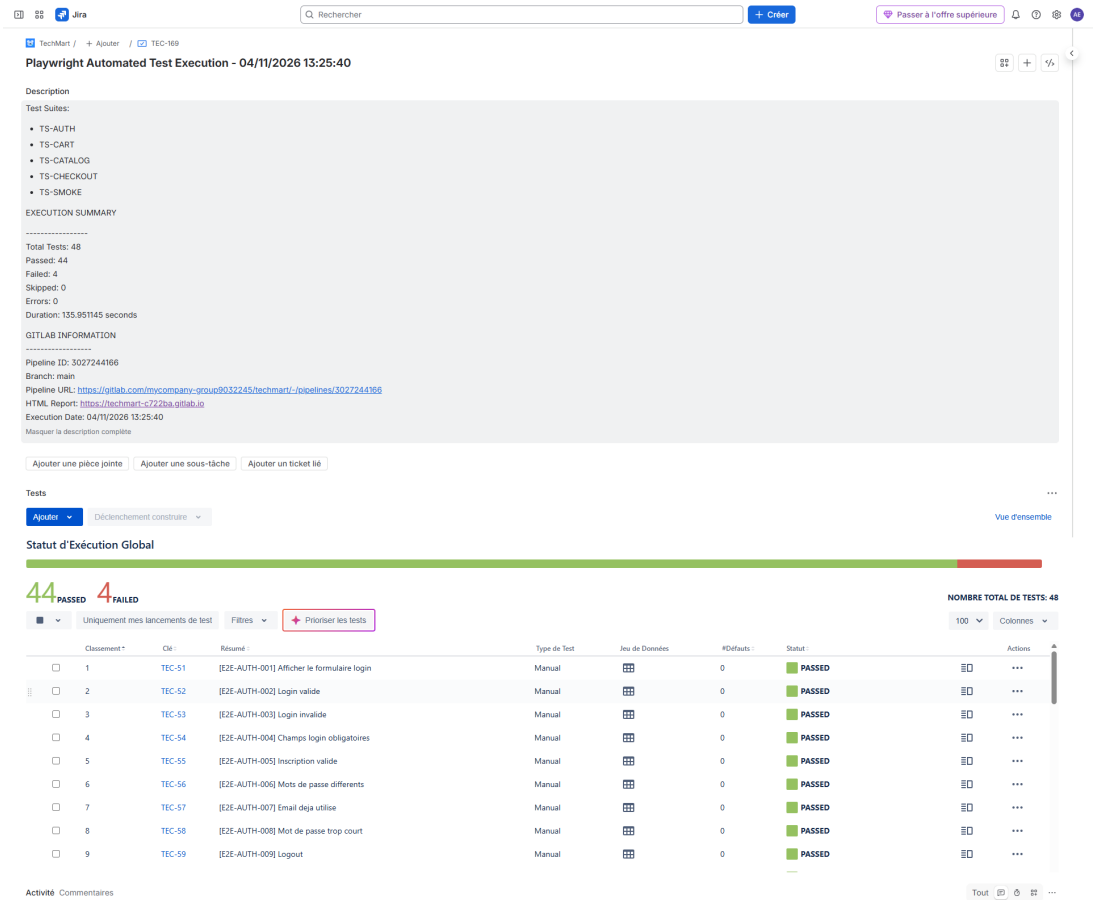


FIGURE 6.3 – Résultats des tests automatisés importés dans une Test Execution Xray.

6.5 Publication du rapport HTML

En plus du rapport JUnit et de l'import dans Xray, Playwright génère un rapport HTML. Ce rapport est publié avec GitLab Pages après l'exécution du pipeline.

Le rapport HTML permet de consulter les résultats de manière visuelle. Il affiche les suites de test, les scénarios réussis ou échoués, ainsi que des informations utiles pour analyser un échec.

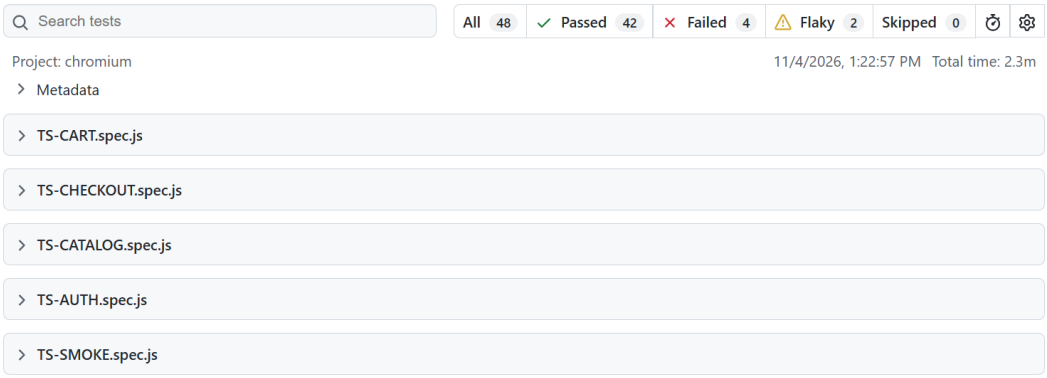


FIGURE 6.4 – Rapport HTML Playwright publié avec GitLab Pages.

6.6 Conclusion

Dans ce chapitre, les résultats Playwright ont été transformés en rapports JUnit et XML. Les fichiers XML ont permis d'importer automatiquement les résultats des tests dans Xray.

Cette étape complète le processus d'automatisation, car elle assure un suivi clair des résultats dans GitLab, Xray et le rapport HTML Playwright.

Chapitre 7

Résultats, anomalies détectées et bilan

7.1 Introduction

Ce chapitre présente les résultats obtenus pendant les tests manuels et automatisés. Il présente aussi les anomalies détectées dans l'application TechMart.

L'objectif est de montrer l'intérêt de la méthode utilisée : les besoins sont définis dans Jira, les cas de test sont gérés dans Xray, les parcours sont automatisés avec Playwright et les résultats sont envoyés automatiquement vers Xray.

7.2 Résultats des tests

Les tests ont permis de vérifier les principales fonctions de TechMart : le catalogue, le panier, l'authentification et le checkout.

Lors d'une exécution manuelle, 48 cas de test ont été exécutés. Parmi eux, 46 tests ont réussi et 2 tests ont échoué. Ces échecs ont permis d'identifier des problèmes liés à la gestion du stock.

Les résultats sont visibles dans Xray. Chaque cas de test possède un statut qui permet de suivre facilement son résultat.

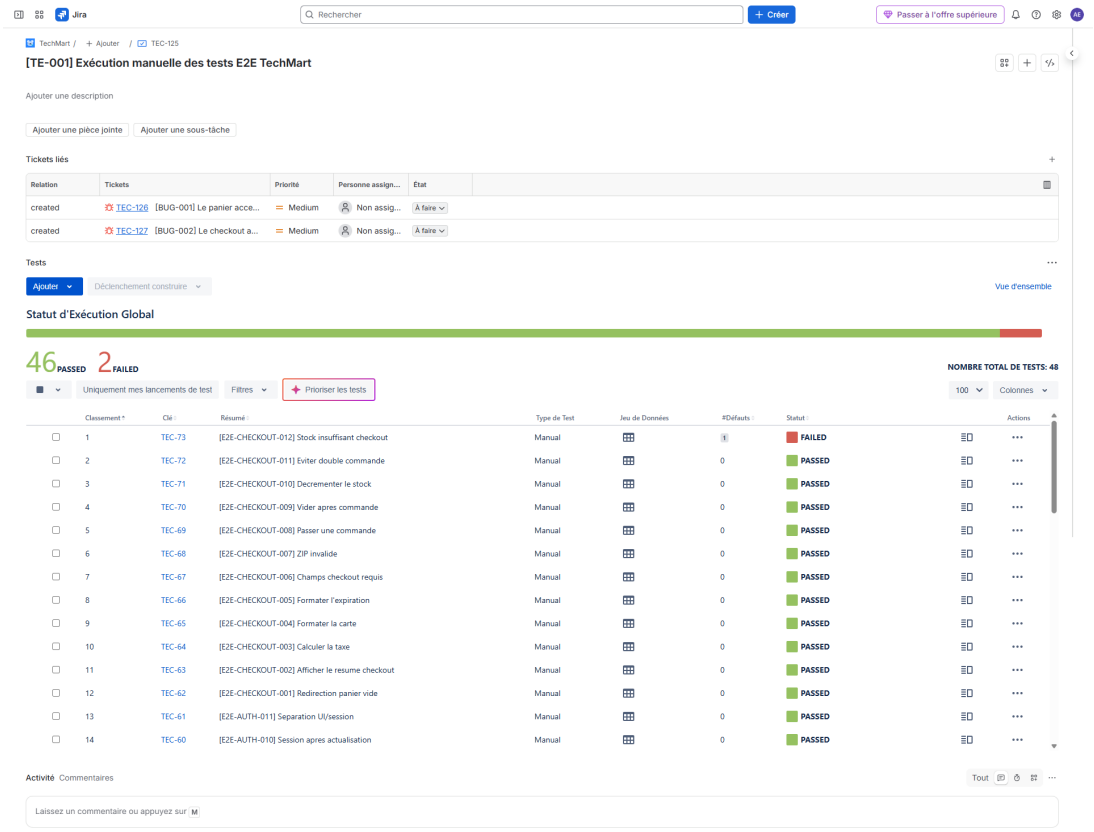


FIGURE 7.1 – Résultats d'une exécution manuelle des 48 cas de test dans Xray.

7.3 Anomalies détectées

Les tests ont permis d'identifier deux anomalies fonctionnelles liées au stock.

La première anomalie concerne le panier. L'application accepte une quantité supérieure au stock disponible. Pourtant, le système devrait empêcher l'utilisateur d'ajouter plus de produits que la quantité disponible.

La deuxième anomalie concerne la validation de commande. L'application accepte une commande même lorsque la quantité demandée dépasse le stock disponible. Le système devrait refuser la commande et afficher un message d'erreur.

Ces anomalies ont été créées sous forme de tickets dans Jira. Elles sont liées aux cas de test concernés afin de garder une trace entre le problème détecté et le scénario de test.

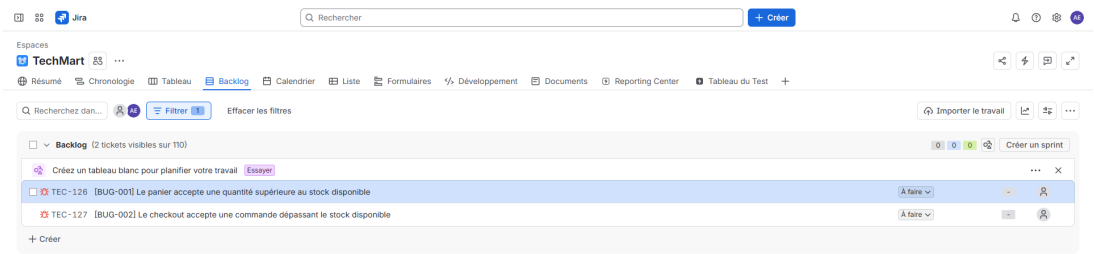


FIGURE 7.2 – Anomalies créées dans Jira à partir des tests en échec.

7.4 Apports du projet

Ce projet m'a permis de découvrir un processus complet de test logiciel.

J'ai appris à analyser les besoins et à créer des User Stories dans Jira. J'ai aussi appris à rédiger des cas de test, à les exécuter dans Xray et à suivre leurs résultats.

L'automatisation avec Playwright m'a permis de comprendre comment simuler les actions d'un utilisateur dans un navigateur. L'utilisation du modèle Page Object Model a permis de mieux organiser le code des tests.

Enfin, GitLab CI/CD a permis de lancer les tests automatiquement et de générer les rapports. L'import des résultats dans Xray a montré l'intérêt de relier les tests automatisés au suivi qualité dans Jira.

7.5 Améliorations possibles

Plusieurs améliorations peuvent être ajoutées dans le futur :

— corriger les anomalies liées à la gestion du stock ;
— ajouter de nouveaux cas de test ;
— exécuter les tests sur plusieurs navigateurs ;
— ajouter des tests de performance et de sécurité ;
— améliorer le reporting et les tableaux de suivi.

7.6 Conclusion

Dans ce chapitre, les résultats des tests et les anomalies détectées ont été présentés. Les tests ont permis de vérifier les fonctions principales de TechMart et d'identifier des problèmes liés au stock.

Le projet a permis de mettre en place un processus complet, depuis les besoins jusqu'au reporting automatique. La conclusion générale présentera le bilan final du projet.

Conclusion générale

Ce projet a permis de mettre en place un processus complet d'automatisation des tests pour l'application e-commerce TechMart.

Le travail a commencé par l'analyse des besoins et la création des User Stories dans Jira. Ensuite, les cas de test ont été rédigés et organisés dans Xray. Cette étape a permis de créer un lien entre les besoins, les tests et les anomalies détectées.

Les principaux parcours utilisateurs ont ensuite été automatisés avec Playwright. Les tests couvrent le catalogue, le panier, l'authentification et la validation des commandes. Le modèle Page Object Model a permis d'organiser le code et de faciliter la maintenance des tests.

L'intégration de GitLab CI/CD a permis de lancer les tests automatiquement dans un pipeline. Les rapports JUnit et XML ont été générés après l'exécution. Les résultats ont ensuite été importés dans Xray afin de suivre les statuts des cas de test dans Jira. Le rapport HTML Playwright a aussi été publié avec GitLab Pages.

Ce projet représente ma première expérience pratique avec Jira, Xray, Playwright et GitLab CI/CD. Il m'a permis de comprendre les différentes étapes d'un processus de test logiciel, depuis l'analyse des besoins jusqu'au reporting automatique.

Enfin, les tests réalisés ont permis d'identifier des anomalies liées à la gestion du stock dans l'application TechMart. Dans le futur, le projet peut être amélioré en corrigeant ces anomalies, en ajoutant de nouveaux scénarios de test et en exécutant les tests sur plusieurs navigateurs.